How the Dev Tools Portal Works

A short guide to what the portal does, who can see what, and how work moves from an ask to something shipped.

What it is

The Dev Tools Portal is one internal site that ties together four things that used to live in separate places: raising a request for something to be built or fixed, tracking that work as it moves through the team, keeping an eye on deployments once it ships, and reporting on how all of it is going. It's built on the same system the rest of the business already runs on, so it inherits the same logins, the same user records, and the same idea of “who is allowed to do what.”

Who sees what

Every page in the portal is switched on for a specific set of people, and the portal decides what to show the moment someone logs in — there's no separate password or list to maintain.

Everyone	Can raise a request and see the status of anything that's been raised — not just their own. Requests are shared and visible to the whole company, not a private inbox.
Development team	Everything above, plus the working pages: their own backlog (now split into a today view, a working list or board, their own requests, and a progress view), the shared team board, code and deployment history, and the deployments page.
Project leads	Everything a requester sees, plus the review queue (deciding what gets taken on), the portfolio dashboard (how everything is tracking), and sign-off on any deployment raised during the business's busiest hours.
Portal settings	Locked to a small admin group only — this is where pages, permissions, and the peak-hours window itself get configured.

If someone lands on a page they're not set up for, the portal quietly takes them back to the page that is theirs — there's no dead end or error screen to explain away.

One more rule worth knowing: only the development team and project leads can add a task straight onto the shared board. Anyone else who tries starts a request instead — it goes through the same review as anything raised through the front door, so nothing lands on the team's plate unreviewed, and the request/priority/module fields that only make sense once someone has taken ownership of the work are filled in by whoever accepts it, not guessed at by whoever asked for it.

The four areas of the portal

1. Requests & tracking

Where work starts, and where anyone can check on something they've asked for.

- **Requests** — raise a request and see the status of anything raised, by anyone.
- **My Backlog** — a developer's personal work: what's due today and across the coming week, the working list or a drag-and-drop board, their own requests gathered in one place, and a look at how their week and their backlog's age are trending.
- **Review Queue** — where project leads look at new requests and decide to schedule, defer, or decline them.
- **Backlog Board** — a shared, drag-and-move board of everything in progress across the team.
- **Portfolio Dashboard** — the roll-up view, now organized into six focused tabs: the week's pulse and what needs attention, delivery speed, who's carrying the load and who's shipping, how old the open backlog is, where the work sits by module, and a forecast of where the backlog is headed.
- **Deployments** — every deployment that's been raised, its current state, and — for anything raised during peak hours — a place for a project lead to approve or hold it before it goes out.

2. Developer tools

Working tools for the people building and maintaining the system day to day — not something the rest of the business needs to open, but useful to know exists.

- **Dev Dashboard** — a developer's personal starting point and summary.
- **Code Editor & Snapshots** — editing and version history for the system's configuration.
- **Hooks Explorer** — a way to search what every installed app is wired to react to.
- **Activity Log** — a record of who changed what, and when.

3. Admin

Reference data and portal configuration — kept separate so it can't be changed by accident.

- **Master Data** — the shared lists the rest of the portal draws on (priorities, categories, and similar).
- **Portal Settings** — controls which pages exist and who can see them, and where the peak-hours window for deployments is configured.

How a request becomes finished work

Every request follows the same path, so nothing depends on someone remembering to chase it up:

1. Raised	2. Under review	3. Approved	4. Scheduled	5. In progress	6. Completed

A project lead reviews what comes in and decides whether it moves forward — approve, defer, or reject. Once approved and scheduled, it becomes an actual piece of tracked work for the team, and its status stays visible the whole way through, to the person who raised it as well as anyone managing the workload. A request can also be withdrawn by whoever raised it at almost any point before it's finished.

Keeping deployments safe during busy hours

The portal can be told when the business is at its busiest — a peak-hours window, set once in Portal Settings. Any deployment raised inside that window is automatically held: it can't move into progress until a project lead has reviewed and approved it. Outside that window, nothing changes — deployments move exactly as they always have. Whether a given deployment needed that sign-off is decided the moment it's raised and doesn't change afterwards, even if the window is edited later or someone tries to push it through the underlying system directly.

Deployments

The portal keeps a full history of what's been deployed, to where, and its outcome (deployed, failed, retried). The dedicated Deployments page is where that history lives day to day — useful for answering “when did this go out” or “has this been released yet” without having to ask, and it's also where a project lead clears anything waiting on peak-hours approval.

The dashboards are now visual

Both the portfolio dashboard and My Backlog moved from plain lists to actual charts where a chart is genuinely the clearer way to see something: bar charts for workload and delivery per module and per person, donut charts wherever a few categories add up to a whole (like how requests get answered, or the mix of task statuses), and — on the portfolio's forecast tab — a chart that plots the backlog's size over recent weeks and projects it forward, using how reliably the team has actually met its own due dates historically, not just a flat guess.